



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

September 2026

CSA0619– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

Student Name (Reg. No.)

Student Name (Reg. No.)

Student Name (Reg. No.)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

B. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques** improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**. The developed solution must determine an effective switching threshold between the two

approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset.**

C. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all pairs with complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω , and Θ notations**, and solve the recurrence using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example, , , , and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data.**

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.
- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance results.
- Perform theoretical efficiency analysis using **Big-O, Ω , and Θ notations** and analyze the

Divide-and-Conquer recurrence using the **Master Theorem**.

- Evaluate the algorithms using increasing input sizes, for example, , , , and **higher input sizes wherever computationally feasible**.
- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the **computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

- Describe the **Closest-Pair-of-Points problem** in the context of real-world geospatial infrastructure planning.
- Obtain an appropriate **publicly available real-world coordinate dataset**.
- Describe the **dataset, attributes, number of records, and source**.
- Define the expected **input and output** of the problem.
- Explain the use of **Euclidean distance** for identifying the closest pair of points.
- Identify the computational challenges associated with **increasing input size**, particularly for the Brute-Force approach.
- State the relevant **assumptions, requirements, and constraints**.

2. Application of Course Knowledge

Identify and apply the relevant concepts:

CO1 – Algorithm Analysis

- Fundamentals of algorithmic problem solving
- Framework for algorithm efficiency analysis
- **Big-O, Ω , and Θ** notations
- Analysis of recursive and non-recursive algorithms
- **Master Theorem** for analyzing the Divide-and-Conquer recurrence

CO2 – Brute Force

- Apply the **Brute-Force Closest-Pair algorithm** by evaluating all possible pairs.
- Analyze its computational efficiency and scalability.

CO3 – Divide and Conquer

- Apply the **Divide-and-Conquer Closest-Pair algorithm** using recursive partitioning about the median x-coordinate and evaluation across the dividing strip.
- Analyze its recursive structure and computational efficiency.

Explain how concepts from **CO1, CO2, and CO3** contribute to the development of the

proposed **Hybrid algorithm**.

3. Solution / Design / Methodology

Design and develop an optimized **Hybrid Closest-Pair algorithm** by appropriately integrating Brute Force and Divide-and-Conquer techniques.

The methodology should follow:

Input Dataset → Data Preparation → Brute-Force Implementation → Divide-and-Conquer Implementation → Hybrid Algorithm Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision

Provide:

- Brute-Force algorithm/pseudocode
- Divide-and-Conquer algorithm/pseudocode
- Developed Hybrid algorithm/pseudocode
- Flowchart/design of the proposed Hybrid approach
- Explanation of the **hybrid switching strategy**
- Method used to determine the **threshold value**
- Mathematical complexity analysis
- Justification of algorithmic design and optimization decisions

4. Use of Modern Tools

Use appropriate tools wherever relevant:

- Python / C / C++ / Java
- Jupyter Notebook / Google Colab / suitable IDE
- Appropriate data-processing, performance-measurement, and visualization libraries

Provide evidence of the **implementation, source code, dataset source, experimental setup, execution outputs, tables, and graphs**.

5. Results and Validation

Experimentally evaluate all three approaches—**Brute Force, Divide and Conquer, and the developed Hybrid algorithm**—using increasing input sizes.

The evaluation should include:

- Execution time
- Number of distance computations/comparisons
- Memory utilization
- Input size
- Scalability

- Theoretical time complexity
- Correctness of the closest-pair result
- Effect of the selected **hybrid threshold**

Present the experimental results using suitable **tables and graphs**.

Compare the performance of the **developed Hybrid algorithm** against the Brute-Force and Divide-and-Conquer approaches using identical input data and the same computational environment.

6. Analysis and Engineering Decision

Interpret the theoretical and experimental results and:

- Compare **Brute Force, Divide and Conquer, and the developed Hybrid algorithm**.
- Analyze the relationship between **theoretical and experimental performance**.
- Evaluate the effect of increasing input size on each approach.
- Analyze the impact of the **switching threshold** on Hybrid algorithm performance.
- Identify the advantages and limitations of each approach.
- Discuss computational and scalability trade-offs.
- Determine the conditions under which each approach performs effectively.
- Justify the **selected threshold value** for the Hybrid algorithm.
- Determine and justify the most suitable approach for **large-scale geospatial infrastructure data**.
- Provide **quantitative experimental evidence** supporting the final engineering decision.

7. Broader Considerations

Discuss how the developed solution contributes to:

- Efficient utilization of computational resources
- Reduction in unnecessary distance computations
- Improved scalability for large geospatial datasets
- Efficient infrastructure planning and analysis
- Practical applicability to real-world systems
- Sustainable and computationally efficient solution development

Relate the developed solution to **SDG 9 – Industry, Innovation and Infrastructure**, particularly its potential contribution to efficient and intelligent infrastructure planning.

8. Conclusion

Summarize:

- Closest-Pair-of-Points problem addressed

- Brute-Force and Divide-and-Conquer approaches implemented
- Developed Hybrid algorithm
- Experimentally determined switching threshold
- Major experimental findings
- Best-performing approach
- Complexity and scalability improvements achieved
- Limitations
- Possible future optimization

9. Student Reflection

Answer individually:

1. What did you learn from developing and optimizing the Hybrid Closest-Pair algorithm beyond implementing the algorithms directly taught in the classroom?
2. What challenges did you encounter while determining the switching threshold and evaluating performance across different input sizes, and how did you address them?
3. If additional computational resources or time were available, how would you further improve the developed algorithm and why?
4. How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

10. References

Cite:

- Dataset source
- Algorithms/research literature consulted
- Software/tools/libraries used
- Other external resources
- AI-assisted tools, if used
- Use an appropriate and consistent citation format.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization of Algorithmic Solution	20	Develops a meaningful algorithmic solution through integration, adaptation, modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	Develops an appropriate solution with meaningful adaptation or improvement, with minor limitations.	Solution demonstrates limited adaptation or development beyond standard implementation.	Merely reproduces an existing algorithm with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement	15	Demonstrates the effectiveness of the	Good performance	Basic comparison is	Improvement is not

, Comparison & Final Decision		developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	comparison and justification with minor limitations.	provided, but improvement or final justification lacks depth.	demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development, SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	15
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action: